

Brian Demers on Observability into the Toolchain - Resumen del podcast

- Entrevistado: **Brian Demers**
- Tema: Observabilidad en el *toolchain* (cadena de herramientas)

Trasfondo de Brian Demers

Se enfoca en **Java**, **OSS** (*Open Source Software*), miembro de **Apache**, contribuye al directio **Maven**.

Definiciones básicas que no aparecen en el podcast

- Build: Proceso durante el desarrollo en el que un programa se compila, se prueba y se genera un ejecutable o archivo final.
- Toolchain: conjunto de herramientas de desarrollo que se ejecutan en secuencia para un fin.
- Observabilidad: Característica de un sistema o software que permite conocer el estado interno del mismo en base a su salida.

Parte 1 (1:10-4:15) Toolchains

Existen varias toolchains (toolchain de compilación, toolchain de build...): ¿a cuáles se refiere Brian? A todas, aunque en el **podcast se centra en introducir la observabilidad en las toolchains de build**.

Concepto de "**Push observability left**": asegurarse de que la observabilidad se utiliza lo antes posible en el desarrollo. Esto arregla problemas como los "**flicky tests**" (tests que a veces pasan y a veces fallan); cuanto antes te des cuenta de ellos, mejor. La observabilidad ayuda a ello porque vas a tener información sobre el sistema antes.

Parte 2 (4:15-9:05) ¿Qué problema intentamos solucionar introduciendo observabilidad en el toolchain?

A veces los **tiempos de build son muy lentos** (2 minutos, 20 minutos), esto reduce la productividad.

Algunos sistemas **solo ejecutan ciertas pruebas en el servidor** durante el ciclo de integración continua y **no permiten ejecutarlas en local**; esto también afecta a la productividad ya que el servidor puede ser lento en ejecutarlas.

Necesitamos que el feedback sea tan rápido como posible. Desde el punto de vista de la observabilidad, **necesitamos tener datos** para saber **cuánto tiempo va a tardar** y dónde están los **cuellos de botella** en la build para mejorar los tiempos.

Parte 3 (9:05-10:25) Observabilidad, pilares básicos

Siempre se tienen en cuenta los **3 pilares de la observabilidad**: se centra en **logs** (herramientas proveen logs, a veces demasiados, ¿cómo los filtras?) y **tracing** (¿dónde están los cuellos de botella en tus builds?)

Normalmente los problemas y cuellos de botella suelen encontrarse en las **pruebas**.

Parte 4 (10:25-13:40) ¿Qué capacidades de observabilidad ofrecen las herramientas actuales?

En general, **no muchas**, pero **se ven mejoras** últimamente.

Aunque no se den herramientas de observabilidad, siempre se pueden hacer cosas como **medir cuánto tiempo tarda cada tarea**; es una gran mejora y mejor que nada. Puedes tomar decisiones en base a esto sobre en qué invertir tiempo.

Algunas **herramientas** dan **demasiada información** (warnings de IDE), ¿cómo los filtramos? Brian dice que la forma de mejorarlo es: o **tratar los warnings como errores** (el desarrollador solo quiere saber si el programa es aceptable o no) o crear algún **sistema que realice un seguimiento los warnings en el tiempo** y vea si se reducen o no.

Si nadie revisa esos datos, nada va a mejorar.

Parte 5 (13:40-14:35) ¿Se aplica la observabilidad en toolchains en equipos actualmente?

Le da especial importancia a *flacky tests*, **hacen perder mucho el tiempo**, sobre todo en organizaciones grandes. Afirma que en este caso la observabilidad es crítica.

Parte 6 (14:35-20:25) ¿Cómo afecta el código generado por LLM a la observabilidad?

El hecho de que más código entre en el sistema requiere **más carga en el proceso de build**: más tiempos de compilación, más tests, más warnings... **más complejidad**.

La observabilidad ayuda en este caso: por ejemplo, si aumentan las líneas de código demasiado, quizás deberíamos dividir el proyecto, ¿es sostenible?. **Todos estos datos son provistos por la observabilidad y ayudan a tomar una decisión**. Otros ejemplos donde se utiliza la observabilidad es la categorización de errores o conocer tiempo de dependencias de descarga de builds en CI.

Parte 7 (20:25-22:06) ¿Conoces algunos clientes que hayan implementado observabilidad debido a LLM?

Sí, debido a un aumento de complejidad. El nuevo sistema ahora se comporta más lento, pero se debería arreglar; la observabilidad ayuda a decidir en qué centrarse.

Parte 8 (22:06-24:30) ¿Cómo de crítico es la procedencia del código en estas organizaciones debido a LLM?

Muy crítica. **Hay mucho interés en OSS**; se debe saber de dónde viene el código para poder respetar las licencias.

Parte 9 (24:30-27:40) ¿Cómo convencer de introducir observabilidad en el toolchain de build?

Es un coste, sí, pero ayuda a la reducción de riesgos.

Parte 10 (27:40-31:00) ¿Cómo deberían gestionarla los equipos?

Cuanta más observabilidad en la toolchain, es más compleja: es **importante mantener el sistema build sencillo**.

Nuestros sistemas builds son muy complejos, hay que simplificarlos, pero **aun así es necesaria la observabilidad**.

Parte 11 (31:00-32:55) Cambios organizacionales y culturales necesarios para observabilidad

"Algo se rompe, no es mi culpa."

Se necesita **educación** y **darle importancia al equipo de CI**.

Parte 12 (32:55-37:35) ¿Qué datos elegir para observar?

Idealmente, todo y luego filtrar; en la práctica, empezar por los datos relacionados con **problemas básicos que te importan**.

¿Cómo enfocarlo en un proyecto greenfield? Utilizar todas las **recolecciones de datos que utilicen tus herramientas**, y también **tomar algunas decisiones en torno a ello**: control de *flaky tests*, **medición de tiempos de build**. También se pueden asociar acciones a los datos de observabilidad: crear nuevo issues, enviar emails...guau me a

Parte 13 (37:35-40:34) Sistemas viejos (legacy)

Normalmente **ejecutar tests** y el **linting** es lo que tarda. Una posible solución: si ya te aparecen los errores en el IDE, no sería necesario ejecutar el linting en la build local.

Los problemas relacionados con la seguridad: en vez de intentar gestionarlos todos de golpe, intentar reducirlos con el tiempo y comprobar esto con la observabilidad.

Parte 14 (40:34-42:45) Recomendaciones de métricas iniciales

Tiempo que tarda la build es la más básica y recomendable.

Parte 15 (42:45-44:40) Problemas de implementación más comunes en organizaciones

Los problemas principales suelen ser de **comunicación** al implementarlo con otros equipos o de pedir permisos a la organización.

Parte 16 (44:40-47:00) Futuro de observabilidad

-Como se dijo al principio del podcast, va mejorando y Brian prevé que tome más importancia en el futuro.